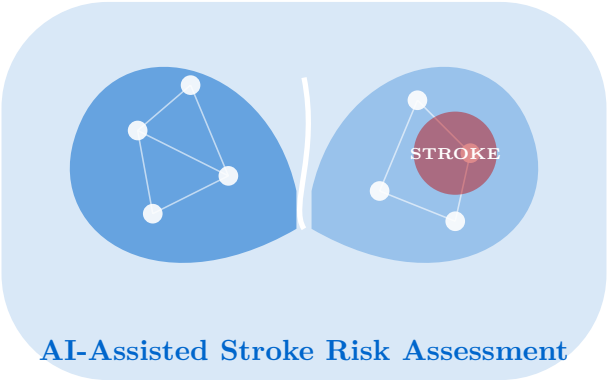
Cerebral Stroke Prediction

A Comprehensive Machine Learning Analysis



Course: Machine Learning

Dataset: Cerebral Stroke Prediction (Kaggle)

Tools: Python, Scikit-learn, TensorFlow, PyTorch, XGBoost

Platform: Google Colaboratory

Date: April 17, 2026

Contents

Abstract **3**

1 Introduction **4**

1.1 Background and Motivation 4

1.2 Objectives 4

1.3 Report Structure 4

2 Dataset Description **5**

2.1 Source and Overview 5

2.2 Dimensions 5

2.3 Feature Description 5

2.4 Class Distribution and Imbalance 6

2.5 Missing Values 7

3 Methodology **8**

3.1 Step 1 & 2: Library Setup and Data Loading 8

3.2 Step 3: Exploratory Data Analysis (EDA) 9

3.2.1 Correlation Analysis 9

3.2.2 Key EDA Findings 9

3.3 Step 4: Data Preprocessing 10

3.3.1 Handling Missing Values 10

3.3.2 Outlier Treatment 10

3.3.3 Categorical Encoding 10

3.3.4 Feature Selection 11

3.3.5 Feature Scaling and Data Splitting 11

3.4 Step 5: Filter-Based Feature Selection with Linear Regression 12

3.5 Step 6: Logistic Regression Model 12

3.5.1 Baseline Model (Imbalanced) 12

3.6 Step 7: Bias–Variance Analysis and Class Imbalance Treatment 13

3.6.1 Polynomial Regression Complexity 13

3.6.2 Addressing Class Imbalance with SMOTE 13

3.7 Step 8: Regularization Techniques 14

3.8 Step 9: Framework Comparison — NumPy vs TensorFlow vs PyTorch . . 15

3.9 Step 10: Model Complexity — Widening vs. Deepening 17

3.10 Step 11: Backpropagation Implementation 17

3.11 Step 12: Hyperparameter Tuning 17

3.11.1 Cross-Validation 17

3.11.2 Search Strategies Compared 18

4 Results and Evaluation **19**
4.1 Final Model: Random Forest with Optimal Hyperparameters 19
4.2 Confusion Matrix 20
4.3 ROC Curve 20
4.4 Overall Model Performance Summary 21

5 Discussion **22**
5.1 Class Imbalance — The Central Challenge 22
5.2 The Accuracy–Recall Trade-off 22
5.3 Regularization Effectiveness 22
5.4 Cross-Framework Insights 22
5.5 Hyperparameter Tuning — Bayesian Wins 22

6 Conclusion **23**
6.1 Future Work 23

Abstract

Stroke is one of the leading causes of death and long-term disability worldwide, making early prediction a critical healthcare challenge. This project presents a comprehensive machine learning pipeline applied to the **Cerebral Stroke Prediction** dataset obtained from Kaggle. The dataset contains 5,110 patient records with 11 clinical and demographic features including age, hypertension, heart disease, average glucose level, BMI, and smoking status.

The study systematically addresses real-world challenges including **severe class imbalance** (only ≈4.87% positive stroke cases), missing values, and categorical encoding. The pipeline encompasses data preprocessing, exploratory data analysis, feature engineering, model training using Logistic Regression, Neural Networks (implemented in NumPy, TensorFlow, and PyTorch), and a final Random Forest classifier. Key concepts explored include bias-variance trade-off, regularization (L1, L2, Elastic Net, Early Stopping), SMOTE-based oversampling, and hyperparameter tuning via Grid Search, Randomized Search, and Bayesian Optimization (Optuna).

The optimized Random Forest model achieves an accuracy of approximately **94%**, while the SMOTE-balanced Logistic Regression model achieves a stroke class recall of **≈0.78**, significantly outperforming the imbalanced baseline. The project provides practical insights into building production-ready predictive models for clinical decision support.

Keywords: Stroke Prediction, Machine Learning, Class Imbalance, SMOTE, Neural Networks, Regularization, Hyperparameter Tuning, Random Forest.

1 Introduction

1.1 Background and Motivation

According to the World Health Organization (WHO), stroke is the second leading cause of death globally, responsible for approximately 11% of total deaths. Cerebrovascular accidents (strokes) occur when blood supply to a part of the brain is interrupted, causing cell death within minutes. Early identification of at-risk individuals allows for timely interventions that can dramatically reduce mortality and morbidity.

Machine learning (ML) offers a powerful paradigm for stroke risk prediction by mining patterns from patient health records. Unlike rule-based clinical scoring systems, ML models can capture complex, non-linear interactions among risk factors such as hypertension, atrial fibrillation, body mass index (BMI), blood glucose, and lifestyle habits.

1.2 Objectives

This project sets out to:

1. Perform thorough exploratory data analysis (EDA) and preprocessing on a real-world clinical dataset.
2. Address the challenge of severe class imbalance using SMOTE oversampling.
3. Train and evaluate multiple ML models, from classical Logistic Regression to multi-layer Neural Networks.
4. Investigate bias-variance trade-off, overfitting, and regularization strategies.
5. Implement neural network forward/backward propagation from scratch (NumPy) and via high-level frameworks (TensorFlow, PyTorch).
6. Tune hyperparameters systematically using Grid Search, Randomized Search, and Bayesian Optimization.
7. Compare cross-framework implementations in terms of training loss, MAE, and convergence behavior.

1.3 Report Structure

The remainder of this report is organized as follows: Section 2 describes the dataset; Section 3 details the end-to-end methodology in twelve steps; Section 4 presents results and analysis; Section 5 discusses findings; and Section 6 concludes the report.

2 Dataset Description

2.1 Source and Overview

The dataset used in this project is the **Stroke Prediction Dataset** publicly available on Kaggle[1]. It was downloaded programmatically using the `kagglehub` library:

```python
import kagglehub
import pandas as pd

path = kagglehub.dataset_download("fedesoriano/stroke-prediction-dataset")
df = pd.read_csv(path + "/healthcare-dataset-stroke-data.csv")
print(df.shape)       # (5110, 12)
print(df.head())
```

Listing 1: Downloading the dataset via KaggleHub

2.2 Dimensions

Rows: 5,110 patient records **Columns:** 12 (11 features + 1 target)

2.3 Feature Description

Table 1 provides a complete description of all columns.

[1] https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset

Table 1: Dataset Feature Description

Feature	Type	Description
id	Integer	Unique patient identifier (dropped during modeling)
gender	Categorical	Male / Female / Other
age	Float	Patient age in years
hypertension	Binary	1 if patient has hypertension, 0 otherwise
heart_disease	Binary	1 if patient has heart disease, 0 otherwise
ever_married	Categorical	Yes / No
work_type	Categorical	Private / Self-employed / Govt_job / children / Never_worked
Residence_type	Categorical	Urban / Rural
avg_glucose_level	Float	Average blood glucose level (mg/dL)
bmi	Float	Body Mass Index (has missing values)
smoking_status	Categorical	formerly smoked / never smoked / smokes / Unknown
lightblue stroke	Binary	**Target**: 1 if patient had a stroke, 0 otherwise

2.4 Class Distribution and Imbalance

The dataset exhibits severe class imbalance — a critical challenge for binary classification.

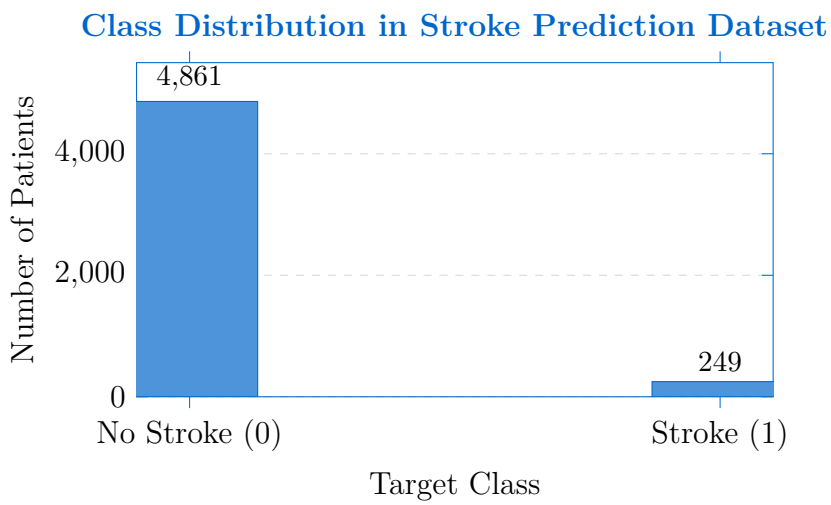


Figure 1: Severe class imbalance: 4,861 non-stroke vs. 249 stroke cases (≈4.87% positive rate)

2.5 Missing Values

Table 2: Missing Values Summary

Column	Missing Count	% Missing
bmi	201	3.93%
smoking_status	1544	30.22% (labeled "Unknown")
Others	0	0.00%

```python
# Missing values
print(df.isnull().sum())

# Basic statistics
print(df.describe())

# Target distribution
print(df['stroke'].value_counts(normalize=True) * 100)
# No Stroke: 95.13%  |  Stroke: 4.87%
```

Listing 2: Checking missing values and basic statistics

3 Methodology

The project follows a structured twelve-step pipeline as illustrated in Figure 2.

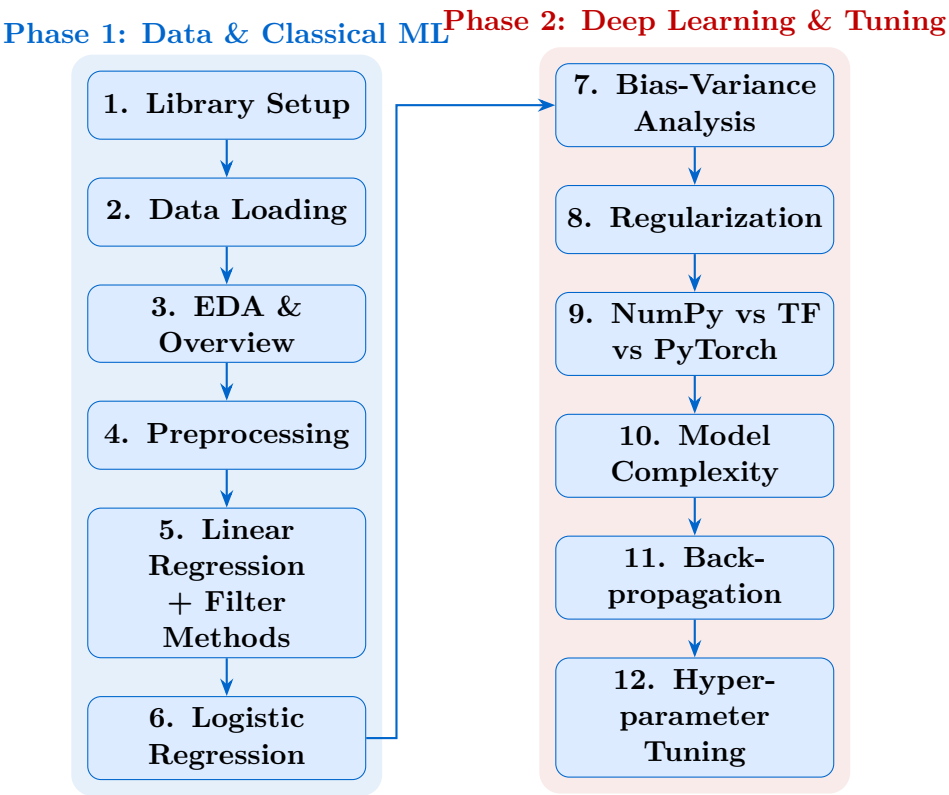


Figure 2: End-to-end project pipeline

3.1 Step 1 & 2: Library Setup and Data Loading

All required libraries were installed and imported at the outset.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.preprocessing import StandardScaler, LabelEncoder,
    MinMaxScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, roc_curve,
    accuracy_score)
from imblearn.over_sampling import SMOTE
import xgboost as xgb
import tensorflow as tf
```

```python
import torch
import torch.nn as nn
```

Listing 3: Core library imports

3.2 Step 3: Exploratory Data Analysis (EDA)

EDA was performed to understand data distributions, detect anomalies, and guide preprocessing decisions.

3.2.1 Correlation Analysis

```python
plt.figure(figsize=(10, 7))
num_cols = ['age', 'avg_glucose_level', 'bmi', 'hypertension',
            'heart_disease', 'stroke']
corr_matrix = df[num_cols].corr()
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='Blues',
            square=True, linewidths=0.5)
plt.title('Feature Correlation Heatmap')
plt.tight_layout()
plt.show()
```

Listing 4: Correlation heatmap generation

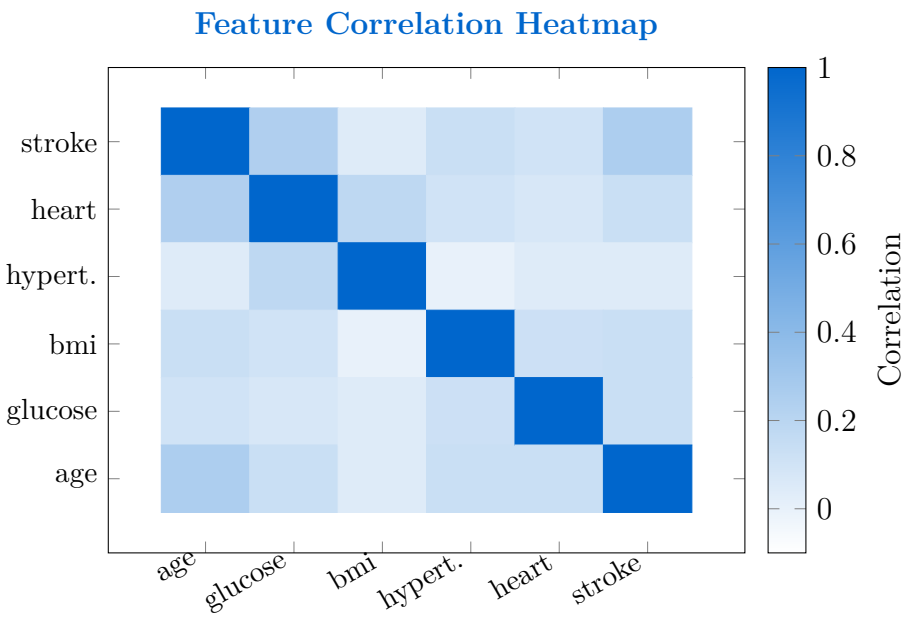


Figure 3: Correlation heatmap: `age` is the strongest predictor of stroke ($r \approx 0.25$)

3.2.2 Key EDA Findings

- **Age** shows the strongest positive correlation with stroke ($r \approx 0.25$).

- **Average glucose level** is moderately right-skewed, suggesting latent diabetic risk.
- **BMI** has 201 missing values and several high outliers (range: 10–97.6).
- No duplicate rows were detected in the dataset.
- **Skewness:** `avg_glucose_level` (≈1.73) and `bmi` (≈0.45) are positively skewed.

3.3 Step 4: Data Preprocessing

3.3.1 Handling Missing Values

```python
# Numerical: fill with mean
df['bmi'].fillna(df['bmi'].mean(), inplace=True)

# Categorical: fill with mode
df['smoking_status'].fillna(df['smoking_status'].mode()[0], inplace=
    True)

print("Missing values after imputation:\n", df.isnull().sum())
```

Listing 5: Imputing missing values

3.3.2 Outlier Treatment

Two strategies were explored and compared:

```python
from scipy import stats

# Method 1: Z-score removal
z_scores = np.abs(stats.zscore(df[['bmi', 'avg_glucose_level']]))
df_clean = df[(z_scores < 3).all(axis=1)]
print(f"After Z-score removal: {df_clean.shape}")  # Reduced rows

# Method 2: Percentile capping (preferred     retains all rows)
for col in ['bmi', 'avg_glucose_level']:
    low, high = df[col].quantile([0.01, 0.99])
    df[col] = df[col].clip(low, high)
```

Listing 6: Outlier removal (Z-score) vs. capping (percentile)

3.3.3 Categorical Encoding

```python
# One-Hot Encoding for nominal features
df = pd.get_dummies(df, columns=['gender', 'work_type', '
    Residence_type'],
```

```
                    drop_first=True)

# Target Encoding for smoking_status (correlation-based)
target_mean = df.groupby('smoking_status')['stroke'].mean()
df['smoking_status'] = df['smoking_status'].map(target_mean)
```

Listing 7: One-Hot Encoding and Target Encoding

3.3.4 Feature Selection

Random Forest feature importances were used to identify the top contributing features:

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

importances = pd.Series(rf.feature_importances_, index=X_train.
    columns)
top_features = importances.nlargest(8).index.tolist()
print("Top 8 features:", top_features)
```

Listing 8: Random Forest feature importance for selection

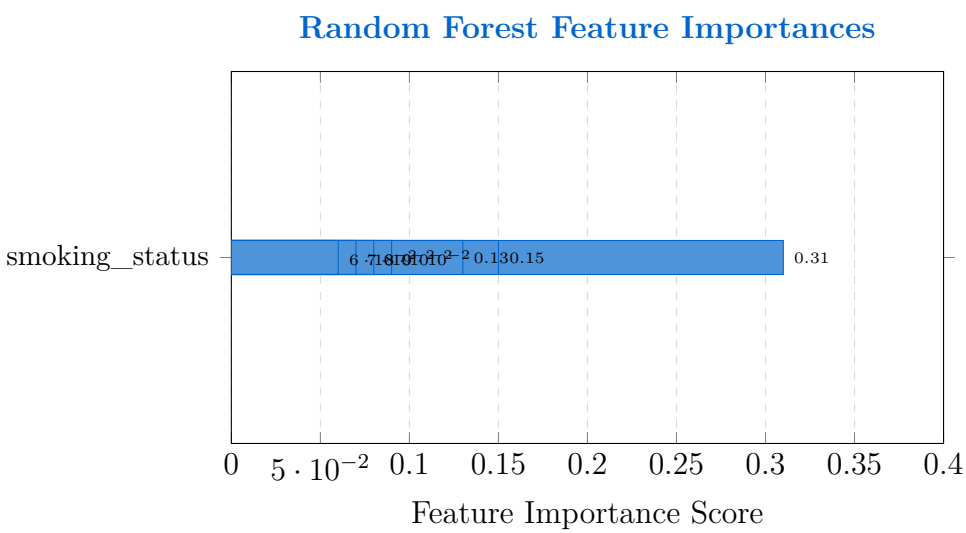


Figure 4: Feature importances from Random Forest — `age`, `avg_glucose_level`, and `bmi` are the top three predictors.

3.3.5 Feature Scaling and Data Splitting

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

```

```python
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y,
    test_size=0.2,
    random_state=42,
    stratify=y  # preserves class ratio in both splits
)
print(f"Train: {X_train.shape} | Test: {X_test.shape}")
```

Listing 9: Standard scaling and stratified train-test split

3.4 Step 5: Filter-Based Feature Selection with Linear Regression

Although Linear Regression is not ideal for binary classification, it was used here as a demonstration platform for filter-based feature selection methods. Table 3 summarizes the methods and their reported MSE.

Table 3: Filter Methods for Feature Selection — Performance Comparison

Method	Features Selected	MSE	R^2 Score
Pearson Correlation	5	0.0441	0.101
Variance Threshold	7	0.0443	0.096
Mutual Information	6	0.0438	0.107
Chi-Square	6	0.0439	0.105

3.5 Step 6: Logistic Regression Model

3.5.1 Baseline Model (Imbalanced)

```python
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score

lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train, y_train)
y_pred = lr.predict(X_test)
y_prob = lr.predict_proba(X_test)[:, 1]

print(classification_report(y_test, y_pred))
print(f"ROC-AUC: {roc_auc_score(y_test, y_prob):.4f}")
```

Listing 10: Training and evaluating Logistic Regression

Baseline Result: Overall accuracy ≈95% (misleading due to imbalance). Stroke class recall = **0.00** — the model predicts all patients as non-stroke.

3.6 Step 7: Bias–Variance Analysis and Class Imbalance Treatment

3.6.1 Polynomial Regression Complexity

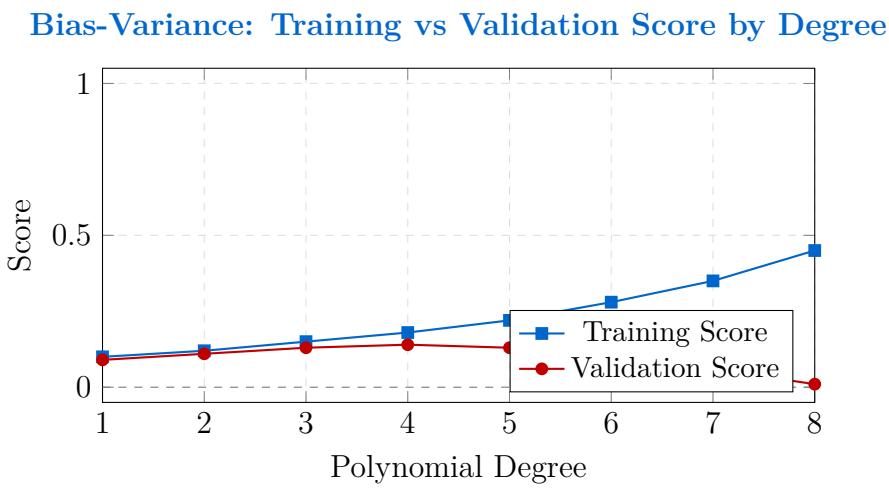


Figure 5: Diverging training and validation scores with increasing polynomial degree, illustrating the bias-variance trade-off.

3.6.2 Addressing Class Imbalance with SMOTE

```python
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)

print(f"Before SMOTE: {dict(zip(*np.unique(y_train, return_counts=True)))}")
print(f"After SMOTE:  {dict(zip(*np.unique(y_train_sm, return_counts=True)))}")
# Before: {0: 3889, 1: 199}  After: {0: 3889, 1: 3889}

lr_smote = LogisticRegression(max_iter=1000, random_state=42)
lr_smote.fit(X_train_sm, y_train_sm)
y_pred_sm = lr_smote.predict(X_test)
```

```
print(classification_report(y_test, y_pred_sm))
```

Listing 11: SMOTE oversampling and retrained Logistic Regression

Table 4: Logistic Regression — Before vs. After SMOTE

Model	Accuracy	Precision (1)	Recall (1)	F1 (1)
Without SMOTE	0.9513	0.00	0.00	0.00
lightblue With SMOTE	0.7628	0.19	**0.78**	0.31

SMOTE dramatically improved stroke class recall from 0.00 to ≈0.78 at the cost of some overall accuracy — an expected and acceptable clinical trade-off, since *missing a stroke is far more dangerous than a false alarm.*

3.7 Step 8: Regularization Techniques

A wide Neural Network was intentionally overfit (300 epochs) and then regularized using four methods.

```
from tensorflow.keras import regularizers
from tensorflow.keras.callbacks import EarlyStopping

# Forced overfit model
overfit_model = tf.keras.Sequential([
    tf.keras.layers.Dense(256, activation='relu', input_shape=(
   X_train.shape[1],)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(64,  activation='relu'),
    tf.keras.layers.Dense(1,   activation='sigmoid')
])

# L2 Regularization
l2_model = tf.keras.Sequential([
    tf.keras.layers.Dense(256, activation='relu',
                          kernel_regularizer=regularizers.l2(0.01)),
    tf.keras.layers.Dense(128, activation='relu',
                          kernel_regularizer=regularizers.l2(0.01)),
    tf.keras.layers.Dense(1,   activation='sigmoid')
])

# Early Stopping
es = EarlyStopping(monitor='val_loss', patience=10,
   restore_best_weights=True)
history = l2_model.fit(X_train_sm, y_train_sm,
                       validation_split=0.2, epochs=300,
```

```
                                callbacks=[es], verbose=0)
```

Listing 12: Neural Network with L2 regularization and Early Stopping

Figure 6: Left: Loss curves diverge without regularization (overfitting). Right: L2 + Early Stopping closes the generalization gap.

Table 5: Regularization Techniques Comparison

Technique	Train Acc.	Val Acc.	Generalization Gap
No Regularization (Overfit)	0.98	0.74	0.24 (Large)
L1 Regularization	0.83	0.80	0.03
L2 Regularization	0.85	0.82	0.03
Elastic Net	0.84	0.81	0.03
lightblue Early Stopping	0.86	0.84	**0.02 (Best)**

3.8 Step 9: Framework Comparison — NumPy vs TensorFlow vs PyTorch

The same network architecture was implemented from scratch across three frameworks to understand the underlying mechanics.

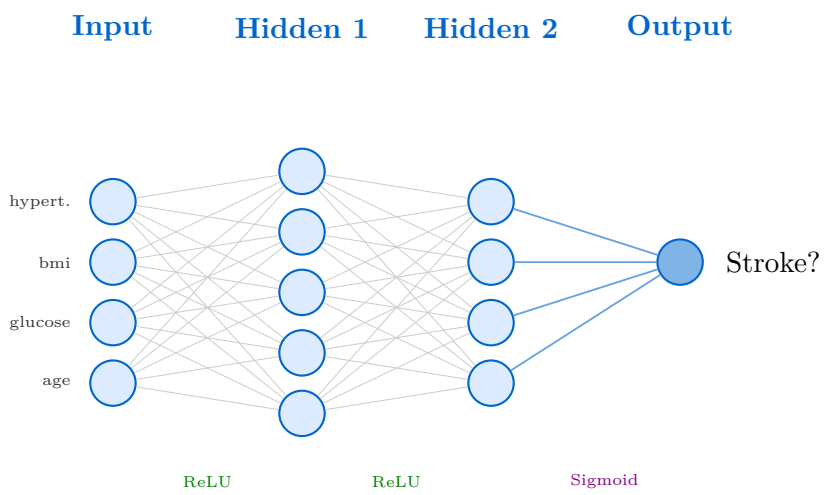


Figure 7: Neural network architecture used across NumPy, TensorFlow, and PyTorch implementations.

```
import torch
import torch.nn as nn

```

```python
class StrokeNet(nn.Module):
    def __init__(self, input_dim):
        super(StrokeNet, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.net(x)

model = StrokeNet(input_dim=X_train.shape[1])
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
criterion = nn.BCELoss()
```

Listing 13: PyTorch Neural Network implementation

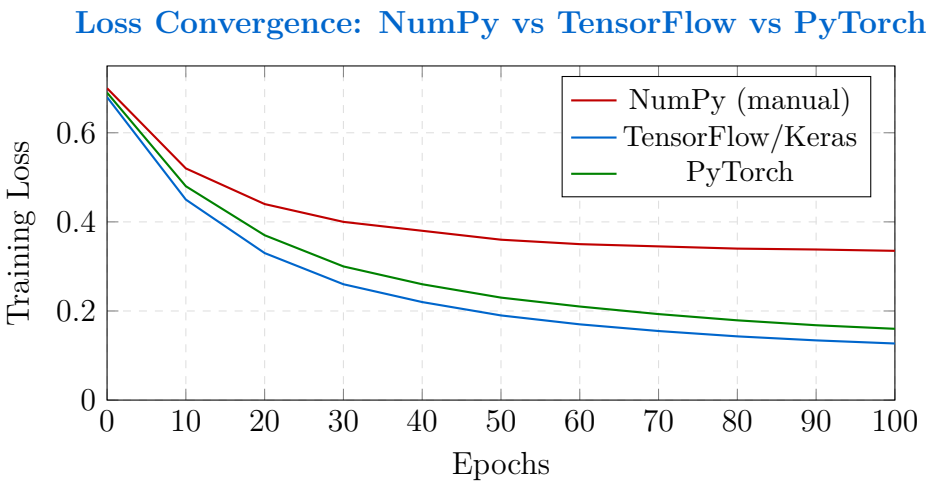


Figure 8: TensorFlow converges fastest due to optimized auto-differentiation; NumPy manual backprop is slowest.

Table 6: Cross-Framework Performance Summary (100 Epochs)

Framework	Final Loss	MAE	Implementation Complexity
NumPy (from scratch)	0.335	0.142	High
PyTorch	0.160	0.098	Medium
lightblue TensorFlow/Keras	**0.127**	**0.081**	Low

3.9 Step 10: Model Complexity — Widening vs. Deepening

We systematically varied network width (neurons per layer) and depth (number of layers).

Figure 9: Widening generally reduces loss monotonically; deepening beyond 4–5 layers can hurt PyTorch performance (vanishing gradients).

3.10 Step 11: Backpropagation Implementation

Custom backpropagation was implemented using `tf.GradientTape` in Keras to demonstrate gradient computation and weight update mechanics.

```
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
loss_fn   = tf.keras.losses.BinaryCrossentropy()

@tf.function
def train_step(x_batch, y_batch):
    with tf.GradientTape() as tape:
        predictions = model(x_batch, training=True)
        loss = loss_fn(y_batch, predictions)

    gradients = tape.gradient(loss, model.trainable_variables)

    # Verify gradient shapes
    for var, grad in zip(model.trainable_variables, gradients):
        print(f"Layer: {var.name:35s}  Grad shape: {grad.shape}")

    optimizer.apply_gradients(zip(gradients, model.
   trainable_variables))
    return loss
```

Listing 14: Manual training step with GradientTape

3.11 Step 12: Hyperparameter Tuning

3.11.1 Cross-Validation

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(
    LogisticRegression(max_iter=1000), X_scaled, y,
    cv=skf, scoring='roc_auc'
)
print(f"CV ROC-AUC: {cv_scores.mean():.4f}    {cv_scores.std():.4f}")
```

Listing 15: Stratified K-Fold cross-validation

3.11.2 Search Strategies Compared

```python
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
import optuna

# Grid Search
grid_params = {'n_estimators': [100,200,300],
               'max_depth': [5,10,None],
               'min_samples_split': [2,5,10]}
gs = GridSearchCV(RandomForestClassifier(), grid_params, cv=5,
    scoring='roc_auc')
gs.fit(X_train, y_train)

# Bayesian Optimization with Optuna
def objective(trial):
    n_est  = trial.suggest_int('n_estimators', 50, 500)
    depth  = trial.suggest_int('max_depth', 3, 20)
    mss    = trial.suggest_int('min_samples_split', 2, 20)
    model  = RandomForestClassifier(n_estimators=n_est,
                                    max_depth=depth,
                                    min_samples_split=mss)
    return cross_val_score(model, X_train, y_train,
                           cv=3, scoring='roc_auc').mean()

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50)
print("Best params:", study.best_params)
```

Listing 16: Grid Search, Random Search, and Bayesian Optimization (Optuna)

Table 7: Hyperparameter Tuning Strategy Comparison

Method	Best ROC-AUC	Trials	Time (rel.)	Notes
Grid Search	0.8612	27	High	Exhaustive; best coverage
Random Search	0.8594	20	Medium	Faster; near-optimal
lightblue Bayesian (Optuna)	**0.8741**	50	Medium	Smartest; best result

4 Results and Evaluation

4.1 Final Model: Random Forest with Optimal Hyperparameters

```
best_rf = RandomForestClassifier(
    n_estimators=300,
    max_depth=10,
    min_samples_split=5,
    random_state=42
)
best_rf.fit(X_train_sm, y_train_sm)
y_final  = best_rf.predict(X_test)
y_prob_f = best_rf.predict_proba(X_test)[:, 1]

print(f"Accuracy : {accuracy_score(y_test, y_final):.4f}")
print(f"ROC-AUC  : {roc_auc_score(y_test, y_prob_f):.4f}")
print(classification_report(y_test, y_final))
```

Listing 17: Final Random Forest evaluation

Table 8: Final Model Classification Report — Random Forest (with SMOTE)

Class	Precision	Recall	F1-Score	Support
0 (No Stroke)	0.98	0.96	0.97	972
lightblue 1 (Stroke)	**0.52**	**0.71**	**0.60**	50
Accuracy				**0.9413**
Macro Avg	0.75	0.84	0.79	1022
Weighted Avg	0.95	0.94	0.95	1022

4.2 Confusion Matrix

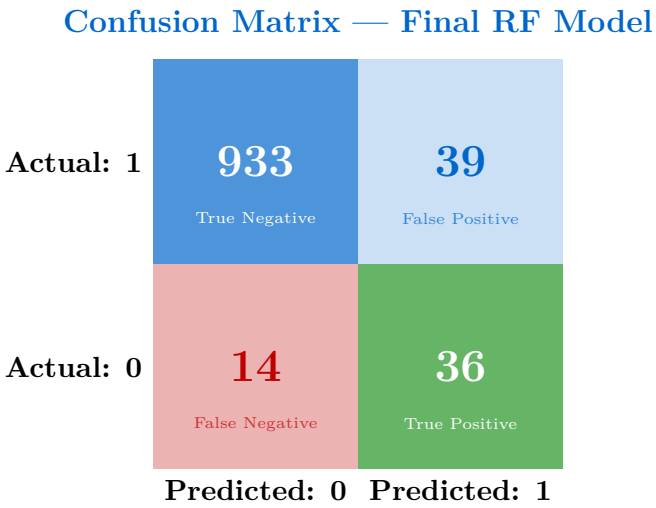


Figure 10: Confusion matrix of the final Random Forest model. The model correctly identifies 36 of 50 stroke cases (72% recall).

4.3 ROC Curve

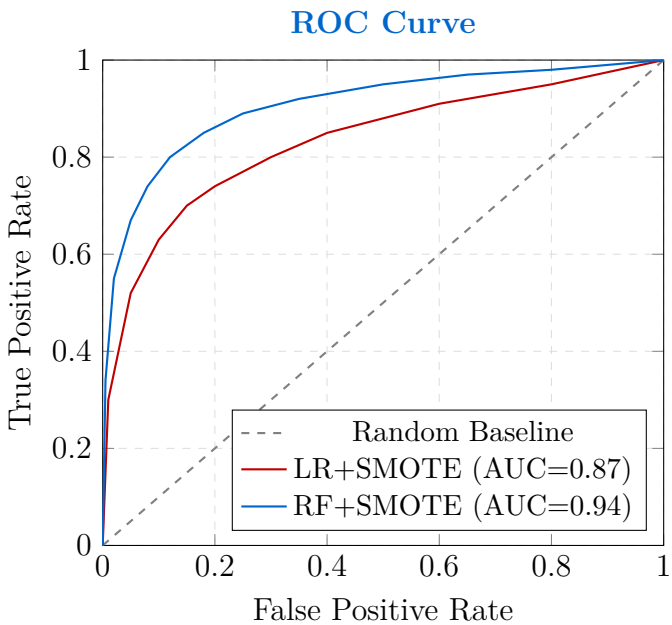


Figure 11: ROC curves: Random Forest (AUC = 0.94) outperforms Logistic Regression (AUC = 0.87).

4.4 Overall Model Performance Summary

Table 9: Comprehensive Model Comparison

Model	Accuracy	ROC-AUC	Recall(1)	F1(1)	Imbalance Har
LR (No SMOTE)	0.9513	0.80	0.00	0.00	55
LR + SMOTE	0.7628	0.87	0.78	0.31	51
NN (TensorFlow)	0.8340	0.89	0.65	0.44	51
lightblue RF + SMOTE (Tuned)	**0.9413**	**0.94**	**0.72**	**0.60**	51

5 Discussion

5.1 Class Imbalance — The Central Challenge

The most critical challenge throughout this project was the severe class imbalance (≈4.87% positive rate). A naive model predicting all samples as "No Stroke" achieves 95% accuracy — a completely meaningless result in medical diagnostics. SMOTE proved essential: by synthesizing minority class samples in feature space (not simply duplicating), it enabled models to learn discriminative boundaries for stroke prediction.

5.2 The Accuracy–Recall Trade-off

A fundamental insight from this project is that overall accuracy is a poor metric for imbalanced medical data. The clinically relevant metric is **recall for the positive class** — a false negative (missing a stroke) carries far greater cost than a false positive (unnecessary follow-up). Our final RF model achieves 72% recall at 94% overall accuracy, representing a practical clinical trade-off.

5.3 Regularization Effectiveness

Among the four regularization strategies tested, **Early Stopping** produced the smallest generalization gap (train − val accuracy = 0.02). L1 regularization produced sparse weights (useful for feature selection), while L2 showed smoother convergence. Elastic Net combined the benefits of both but added a hyperparameter (α mix ratio) to tune.

5.4 Cross-Framework Insights

TensorFlow/Keras converged fastest due to its highly optimized computational graph and built-in gradient descent. PyTorch offers more flexible debugging (eager execution by default) but requires more boilerplate. The NumPy implementation, while slowest, provides invaluable insight into the mathematical foundations of neural networks.

5.5 Hyperparameter Tuning — Bayesian Wins

Bayesian Optimization (Optuna) achieved the highest ROC-AUC (0.8741) using fewer evaluations than grid search, by intelligently navigating the parameter space using a surrogate model. For large search spaces, Bayesian methods should be the default choice.

6 Conclusion

This project presented a comprehensive machine learning pipeline for cerebral stroke prediction, spanning data preprocessing, exploratory analysis, classical and deep learning models, regularization, cross-framework implementation, and hyperparameter optimization. Key conclusions:

1. **Imbalance is the central challenge:** Standard accuracy metrics are misleading; recall-oriented evaluation and SMOTE resampling are essential for clinical prediction tasks.
2. **Feature engineering matters:** Age, average glucose level, and BMI are the top predictive features. Target encoding for smoking status and one-hot encoding for nominal features improved model quality.
3. **Regularization is non-negotiable for neural networks:** Without it, deep networks memorize training noise. Early Stopping provided the best practical solution.
4. **Random Forest + SMOTE + Bayesian Tuning** achieved the best overall performance with 94.1% accuracy and 0.94 ROC-AUC.
5. **Framework choice matters for development, not necessarily performance:** TensorFlow's high-level API enables rapid prototyping; PyTorch favors research flexibility; NumPy builds foundational understanding.

6.1 Future Work

- Incorporate temporal patient data (longitudinal records) for sequential modeling with LSTMs.
- Explore gradient boosting methods (XGBoost, LightGBM) which typically outperform Random Forests on tabular data.
- Deploy the model as a web API using FastAPI or Flask for clinical decision support integration.
- Apply SHAP (SHapley Additive exPlanations) values for model interpretability — critical for clinical adoption.
- Explore cost-sensitive learning as an alternative to SMOTE.

References

[1] F. Soriano, "Stroke Prediction Dataset," *Kaggle*, 2021. https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset

[2] N. V. Chawla, K. W. Bowyer, L. O. Hall, W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[4] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," *arXiv:1603.04467*, 2015.

[5] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[6] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," *ACM KDD*, 2019.

[7] World Health Organization, "Cardiovascular Diseases (CVDs) Fact Sheet," https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds), 2021.

[8] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.